

tpcc_dbfs_vs_postgres_basic_comparison

May 30, 2026

1 TPCC Comparison: dbfs vs PostgreSQL

This notebook compares two BenchBase TPCC runs collected under the same nominal settings and grouped under a shared comparison label:

- comparison label: 20260503
- artifact selection: latest BenchBase artifact prefix under each engine directory
- dbfs result directory: `benchmarking/results/20260503/tpcc/dbfs`
- PostgreSQL result directory: `benchmarking/results/20260503/tpcc/postgres`

The notebook reads summary, metrics, params, samples, results, raw traces, per-procedure result windows, and optional query-trace CSVs from those directories to build side-by-side comparison tables and performance plots.

```
[1]: from pathlib import Path
import json
import re

import matplotlib.pyplot as plt
import pandas as pd
from IPython.display import display

plt.style.use('ggplot')
pd.set_option('display.float_format', lambda value: f'{value:,.3f}')

comparison_root = Path(
    '/Users/yohasebe/Documents/dbfs/benchmarking/results/20260530-122133/tpcc')

# Query trace viewer (early stage - works without metrics.json)
#     dbfs_query_trace.csv

results_root = Path('/Users/yohasebe/Documents/dbfs/benchmarking/results')

#
latest_timestamp = None
latest_trace_path = None

for timestamp_dir in sorted(results_root.glob('**/tpcc/dbfs'), reverse=True):
    trace_path = timestamp_dir / 'dbfs_query_trace.csv'
```

```

if trace_path.exists():
    latest_timestamp = timestamp_dir.parent.parent.name
    latest_trace_path = trace_path
    break

if latest_trace_path:
    print(f"Loading query trace from: {latest_trace_path}")
    query_trace_df = pd.read_csv(latest_trace_path)

    # total_elapsed_us
    query_trace_sorted = query_trace_df.sort_values('total_elapsed_us',
                                                    ascending=False)

    # us ms
    display_df = query_trace_sorted.copy()
    display_df['total_elapsed_ms'] = display_df['total_elapsed_us'] / 1000
    display_df['mean_elapsed_ms'] = display_df['mean_elapsed_us'] / 1000
    display_df['max_elapsed_ms'] = display_df['max_elapsed_us'] / 1000

    display(display_df[[
        'kind', 'shape_key', 'executions', 'total_elapsed_ms',
        'mean_elapsed_ms', 'max_elapsed_ms', 'failures', 'last_error_type'
    ]])
else:
    print("No dbfs_query_trace.csv found in recent results")

```

Loading query trace from: /Users/yohasebe/Documents/dbfs/benchmarking/results/20260530-122133/tpcc/dbfs/dbfs_query_trace.csv

	kind	shape_key	executions	\
0	QUERY	SELECT COUNT(DISTINCT (S_I_ID)) AS STOCK_COUNT...	23	
1	QUERY	SELECT C_FIRST, C_MIDDLE, C_ID, C_STREET_1, C_...	158	
2	QUERY	SELECT S_QUANTITY, S_DATA, S_DIST_01, S_DIST_0...	2279	
3	UPDATE	UPDATE order_line SET OL_DELIVERY_D = ? WHERE ...	20	
4	QUERY	SELECT I_PRICE, I_NAME , I_DATA FROM item WHER...	2281	
5	QUERY	SELECT OL_I_ID, OL_SUPPLY_W_ID, OL_QUANTITY, O...	30	
6	UPDATE	UPDATE stock SET S_QUANTITY = ? , S_YTD = S_YT...	2262	
7	UPDATE	INSERT INTO order_line (OL_O_ID, OL_D_ID, OL_W...	2262	
8	QUERY	SELECT SUM(OL_AMOUNT) AS OL_TOTAL FROM order_l...	20	
9	QUERY	SELECT C_DISCOUNT, C_LAST, C_CREDIT FROM custo...	226	
10	UPDATE	UPDATE district SET D_YTD = D_YTD + ? WHERE D_...	234	
11	QUERY	SELECT W_STREET_1, W_STREET_2, W_CITY, W_STATE...	234	
12	QUERY	SELECT D_STREET_1, D_STREET_2, D_CITY, D_STATE...	234	
13	UPDATE	UPDATE warehouse SET W_YTD = W_YTD + ? WHERE W...	234	
14	QUERY	SELECT O_ID, O_CARRIER_ID, O_ENTRY_D FROM oord...	30	
15	QUERY	SELECT D_NEXT_O_ID, D_TAX FROM district WHERE ...	226	
16	QUERY	SELECT W_TAX FROM warehouse WHERE W_ID = ?	226	
17	UPDATE	UPDATE customer SET C_BALANCE = ?, C_YTD_PAYME...	211	

18	UPDATE	UPDATE district SET D_NEXT_O_ID = D_NEXT_O_ID ...	226
19	QUERY	SELECT C_FIRST, C_MIDDLE, C_LAST, C_STREET_1, ...	106
20	UPDATE	INSERT INTO history (H_C_D_ID, H_C_W_ID, H_C_I...	234
21	UPDATE	INSERT INTO oorder (O_ID, O_D_ID, O_W_ID, O_C...	226
22	UPDATE	INSERT INTO new_order (NO_O_ID, NO_D_ID, NO_W...	226
23	QUERY	SELECT NO_O_ID FROM new_order WHERE NO_D_ID = ...	20
24	QUERY	SELECT O_C_ID FROM oorder WHERE O_ID = ? AND O...	20
25	QUERY	SELECT C_DATA FROM customer WHERE C_W_ID = ? A...	23
26	UPDATE	DELETE FROM new_order WHERE NO_O_ID = ? AND NO...	20
27	QUERY	SELECT D_NEXT_O_ID FROM district WHERE D_W_ID ...	23
28	UPDATE	UPDATE customer SET C_BALANCE = ?, C_YTD_PAYME...	23
29	UPDATE	UPDATE oorder SET O_CARRIER_ID = ? WHERE O_ID ...	20

	total_elapsed_ms	mean_elapsed_ms	max_elapsed_ms	failures	\
0	19,358.632	841.679	1,268.616	0	
1	7,074.403	44.774	315.115	0	
2	5,087.783	2.232	92.237	0	
3	4,954.498	247.724	485.049	0	
4	4,769.666	2.091	54.415	0	
5	3,380.277	112.675	418.344	0	
6	3,112.130	1.375	40.389	0	
7	1,831.257	0.809	82.585	0	
8	1,117.057	55.852	102.242	0	
9	1,107.871	4.902	491.882	0	
10	566.454	2.420	227.021	0	
11	563.907	2.409	68.981	0	
12	549.687	2.349	59.062	0	
13	484.450	2.070	81.936	0	
14	479.652	15.988	48.582	0	
15	477.199	2.111	21.079	0	
16	476.698	2.109	18.175	0	
17	389.206	1.844	12.526	0	
18	313.784	1.388	8.584	0	
19	240.149	2.265	10.168	0	
20	216.107	0.923	5.613	0	
21	210.928	0.933	6.452	0	
22	169.605	0.750	20.291	0	
23	93.743	4.687	13.461	0	
24	87.788	4.389	51.863	0	
25	59.888	2.603	13.453	0	
26	41.928	2.096	9.027	0	
27	37.019	1.609	2.276	0	
28	36.537	1.588	3.637	0	
29	31.271	1.563	6.771	0	

	last_error_type
0	NaN
1	NaN

2	NaN
3	NaN
4	NaN
5	NaN
6	NaN
7	NaN
8	NaN
9	NaN
10	NaN
11	NaN
12	NaN
13	NaN
14	NaN
15	NaN
16	NaN
17	NaN
18	NaN
19	NaN
20	NaN
21	NaN
22	NaN
23	NaN
24	NaN
25	NaN
26	NaN
27	NaN
28	NaN
29	NaN

```
[2]: run_configs = {
    'dbfs': {
        'label': 'tpcc_2026-05-22_06-34-29',
        'display_name': 'dbfs',
        'result_dir': comparison_root / 'dbfs',
    },
    'postgres': {
        'label': 'tpcc_2026-05-22_06-34-29',
        'display_name': 'PostgreSQL',
        'result_dir': comparison_root / 'postgres',
    },
}

configured_ratios = pd.Series(
    {
        'NewOrder': 45.0,
        'Payment': 43.0,
        'OrderStatus': 4.0,
```

```

        'Delivery': 4.0,
        'StockLevel': 4.0,
    },
    name='Configured Ratio (%)')

required_suffixes = {
    'summary': '.summary.json',
    'metrics': '.metrics.json',
    'params': '.params.json',
    'results': '.results.csv',
    'samples': '.samples.csv',
    'raw': '.raw.csv',
}

query_trace_filenames = {
    'dbfs': 'dbfs_query_trace.csv',
    'postgres': 'postgres_query_trace.csv',
}

procedure_suffix = '.results.'

```

```

[3]: def load_terminal_log(result_dir):
    for filename in [
        'benchbase.log', 'benchbase_setup.log', 'benchbase_execute.log'
    ]:
        path = result_dir / filename
        if path.exists():
            return path.read_text(encoding='utf-8')
    return ''

run_configs['dbfs']['terminal_log'] = load_terminal_log(
    run_configs['dbfs']['result_dir'])
run_configs['postgres']['terminal_log'] = load_terminal_log(
    run_configs['postgres']['result_dir'])

field_patterns = {
    'workload': re.compile(r'Benchmark:\s+(\w+)'),
    'dbms_type': re.compile(r'Type:\s+(\w+)'),
    'driver': re.compile(r'Driver:\s+(.+)'),
    'url': re.compile(r'URL:\s+(.+)'),
    'isolation': re.compile(r'Isolation:\s+(\S+)'),
    'loader_threads': re.compile(r'Loader Threads:\s+(\d+)'),
    'scale_factor': re.compile(r'Scale Factor:\s+([\d.]+)'),
    'terminals': re.compile(r'Terminals:\s+(\d+)'),
}

phase_pattern = re.compile(

```

```

    r'PHASE START :: .*\[Time=(\d+)\] \[WarmupTime=(\d+)\] \[Rate=(\d.+)\] .
    ↪*\[Ratios=\[(\d+)\]\]'
)
rate_pattern = re.compile(
    r'measuredRequests=(\d+)\) = (\d.+)\) requests/sec \((throughput\), (\d.+)\)
    ↪requests/sec \((goodput\)'
)

def parse_terminal_log(log_text):
    parsed = {}
    for key, pattern in field_patterns.items():
        match = pattern.search(log_text)
        parsed[key] = match.group(1).strip() if match else None

    phase_match = phase_pattern.search(log_text)
    if phase_match:
        parsed['phase_time_seconds'] = int(phase_match.group(1))
        parsed['warmup_time_seconds'] = int(phase_match.group(2))
        parsed['target_rate'] = float(phase_match.group(3))
        parsed['configured_ratio_list'] = [
            float(value.strip()) for value in phase_match.group(4).split(',')
        ]

    rate_match = rate_pattern.search(log_text)
    if rate_match:
        parsed['terminal_measured_requests'] = int(rate_match.group(1))
        parsed['terminal_throughput'] = float(rate_match.group(2))
        parsed['terminal_goodput'] = float(rate_match.group(3))

    return parsed

parsed_configs = pd.DataFrame([
    {'engine': engine,
     **parse_terminal_log(config['terminal_log'])}
    for engine, config in run_configs.items()])
parsed_configs

```

```

[3]:      engine workload dbms_type                                driver \
0      dbfs      TPCC  POSTGRES                                dev.yohaku.dbfs.jdbc.DbfsDriver
1 postgres      TPCC  POSTGRES  dev.yohaku.dbfs.jdbc.TracingPostgresDriver

                                url \
0                                jdbc:dbfs://dbfs:25432/benchbase
1 jdbc:trace:postgresql://postgres:5432/benchbas...

```

	isolation	loader_threads	scale_factor	terminals	\
0	TRANSACTION_SERIALIZABLE	2	1.0	1	
1	TRANSACTION_SERIALIZABLE	2	1.0	1	

	terminal_measured_requests	terminal_throughput	terminal_goodput
0	NaN	NaN	NaN
1	27,562.000	459.366	457.433

```
[4]: def find_latest_artifact_prefix(result_dir):
    summary_suffix = required_suffixes['summary']
    matches = sorted(result_dir.glob(f'*{summary_suffix}'))
    if not matches:
        raise FileNotFoundError(
            f'No artifact ending with {summary_suffix} under {result_dir}')
    latest_summary = matches[-1]
    return latest_summary.name[:-len(summary_suffix)]

def find_artifact(result_dir, run_label, suffix):
    artifact_path = result_dir / f'{run_label}{suffix}'
    if not artifact_path.exists():
        raise FileNotFoundError(
            f'No artifact ending with {suffix} for {run_label} under {result_dir}')
    return artifact_path

def load_optional_query_trace(engine, result_dir):
    trace_path = result_dir / query_trace_filenames[engine]
    if not trace_path.exists():
        return trace_path, None
    return trace_path, pd.read_csv(trace_path)

def load_run_artifacts(engine, config):
    result_dir = config['result_dir']
    detected_label = find_latest_artifact_prefix(result_dir)
    artifacts = {
        name: find_artifact(result_dir, detected_label, suffix)
        for name, suffix in required_suffixes.items()
    }
    procedure_paths = sorted(
        result_dir.glob(f'{detected_label}{procedure_suffix}*.csv'))
    query_trace_path, query_trace_df = load_optional_query_trace(
        engine, result_dir)
```

```

with artifacts['summary'].open() as handle:
    summary = json.load(handle)
with artifacts['metrics'].open() as handle:
    metrics = json.load(handle)
with artifacts['params'].open() as handle:
    params = json.load(handle)

results_df = pd.read_csv(artifacts['results'])
samples_df = pd.read_csv(artifacts['samples'])
raw_df = pd.read_csv(artifacts['raw'])
procedure_dfs = {
    path.stem.split('.results.', 1)[1]: pd.read_csv(path)
    for path in procedure_paths
}

return {
    'label': detected_label,
    'artifacts': artifacts,
    'summary': summary,
    'metrics': metrics,
    'params': params,
    'results_df': results_df,
    'samples_df': samples_df,
    'raw_df': raw_df,
    'procedure_dfs': procedure_dfs,
    'query_trace_path': query_trace_path,
    'query_trace_df': query_trace_df,
}

runs = {}
validation_rows = []
for engine, config in run_configs.items():
    run_data = load_run_artifacts(engine, config)
    runs[engine] = {**config, **run_data}
    validation_rows.extend([
        {
            'engine': engine,
            'artifact': 'results.csv',
            'rows': len(run_data['results_df']),
            'columns': ', '.join(run_data['results_df'].columns),
        },
        {
            'engine': engine,
            'artifact': 'samples.csv',
            'rows': len(run_data['samples_df']),
            'columns': ', '.join(run_data['samples_df'].columns),
        },
    ])

```

```

    },
    {
        'engine': engine,
        'artifact': 'raw.csv',
        'rows': len(run_data['raw_df']),
        'columns': ', '.join(run_data['raw_df'].columns),
    },
    {
        'engine': engine,
        'artifact': 'procedure_results',
        'rows': len(run_data['procedure_dfs']),
        'columns': ', '.join(sorted(run_data['procedure_dfs'].keys())),
    },
    {
        'engine':
            engine,
        'artifact':
            query_trace_filenames[engine],
        'rows':
            0 if run_data['query_trace_df'] is None else len(
                run_data['query_trace_df']),
        'columns':
            'missing (optional)' if run_data['query_trace_df'] is None else
            ', '.join(run_data['query_trace_df'].columns),
    },
])

```

```

validation_df = pd.DataFrame(validation_rows)
validation_df

```

```

[4]:
   engine      artifact  rows \
0   dbfs      results.csv    12
1   dbfs      samples.csv    60
2   dbfs        raw.csv   532
3   dbfs  procedure_results     5
4   dbfs  dbfs_query_trace.csv    30
5  postgres      results.csv    12
6  postgres      samples.csv    60
7  postgres        raw.csv  27562
8  postgres  procedure_results     5
9  postgres  postgres_query_trace.csv    69

                                     columns
0  Time (seconds), Throughput (requests/second), ...
1  Time (seconds), Requests, Throughput (requests...
2  Transaction Type Index, Transaction Name, Star...
3  Delivery, NewOrder, OrderStatus, Payment, Stoc...

```

```

4 kind, shape_key, sample_sql, executions, total...
5 Time (seconds), Throughput (requests/second), ...
6 Time (seconds), Requests, Throughput (requests...
7 Transaction Type Index, Transaction Name, Star...
8 Delivery, NewOrder, OrderStatus, Payment, Stoc...
9 kind, shape_key, sample_sql, executions, total...

```

```

[5]: summary_rows = []
for engine, run in runs.items():
    parsed = parse_terminal_log(run['terminal_log'])
    summary = run['summary']
    latency = summary['Latency Distribution']

    summary_rows.append({
        'Engine':
            run['display_name'],
        'Run Label':
            run['label'],
        'Measured Requests':
            summary['Measured Requests'],
        'Elapsed Seconds':
            summary['Elapsed Time (nanoseconds)'] / 1_000_000_000,
        'Throughput (req/s)':
            summary['Throughput (requests/second)'],
        'Goodput (req/s)':
            summary['Goodput (requests/second)'],
        'Requests per Minute':
            summary['Throughput (requests/second)] * 60,
        'Throughput - Goodput Gap':
            summary['Throughput (requests/second)'] -
            summary['Goodput (requests/second)'],
        'Average Latency (ms)':
            latency['Average Latency (microseconds)'] / 1000,
        'Median Latency (ms)':
            latency['Median Latency (microseconds)'] / 1000,
        'P95 Latency (ms)':
            latency['95th Percentile Latency (microseconds)'] / 1000,
        'P99 Latency (ms)':
            latency['99th Percentile Latency (microseconds)'] / 1000,
        'Max Latency (ms)':
            latency['Maximum Latency (microseconds)'] / 1000,
        'Terminal Throughput (req/s)':
            parsed.get('terminal_throughput'),
        'Terminal Goodput (req/s)':
            parsed.get('terminal_goodput'),
    })

```

```
summary_comparison_df = pd.DataFrame(summary_rows).set_index('Engine')
summary_comparison_df
```

```
[5]:
```

	Run Label	Measured Requests	Elapsed Seconds	\
Engine				
dbfs	tpcc_2026-05-30_03-28-55	532	60.000	
PostgreSQL	tpcc_2026-05-30_03-31-38	27562	60.000	

	Throughput (req/s)	Goodput (req/s)	Requests per Minute	\
Engine				
dbfs	8.867	8.133	532.000	
PostgreSQL	459.366	457.433	27,561.958	

	Throughput - Goodput Gap	Average Latency (ms)	\
Engine			
dbfs	0.733	112.541	
PostgreSQL	1.933	2.160	

	Median Latency (ms)	P95 Latency (ms)	P99 Latency (ms)	\
Engine				
dbfs	59.975	454.955	1,003.222	
PostgreSQL	1.696	5.294	8.977	

	Max Latency (ms)	Terminal Throughput (req/s)	\
Engine			
dbfs	1,303.534	NaN	
PostgreSQL	187.594	459.366	

	Terminal Goodput (req/s)
Engine	
dbfs	NaN
PostgreSQL	457.433

```
[6]: transaction_names = [
        'NewOrder', 'Payment', 'OrderStatus', 'Delivery', 'StockLevel'
    ]
transaction_pattern = re.compile(r'procedures\.(\\w+)/\\d+\\s+\\[\\s*(\\d+)\\]')

def extract_histogram_counts(log_text, section_name):
    section_pattern = re.compile(
        rf'{section_name}:\\n(.*)?(?:\\n\\w[\\w ]+:|\\Z)',
        re.DOTALL,
    )
    match = section_pattern.search(log_text)
    if not match:
        return {name: 0 for name in transaction_names}
```

```

block = match.group(1).strip()
if '<EMPTY>' in block:
    return {name: 0 for name in transaction_names}

counts = {name: 0 for name in transaction_names}
for name, count in transaction_pattern.findall(block):
    counts[name] = int(count)
return counts

mix_rows = []
for engine, run in runs.items():
    completed = extract_histogram_counts(run['terminal_log'],
                                         'Completed Transactions')
    aborted = extract_histogram_counts(run['terminal_log'],
                                       'Aborted Transactions')
    completed_total = sum(completed.values())
    aborted_total = sum(aborted.values())

    for name in transaction_names:
        mix_rows.append({
            'Engine':
                run['display_name'],
            'Transaction':
                name,
            'Completed':
                completed[name],
            'Aborted':
                aborted[name],
            'Observed Mix (%)': (completed[name] / completed_total *
                                100) if completed_total else 0.0,
            'Abort Rate (%)':
                (aborted[name] / (completed[name] + aborted[name]) * 100) if
                (completed[name] + aborted[name]) else 0.0,
        })

transaction_mix_df = pd.DataFrame(mix_rows)
transaction_mix_df

```

```

[6]:
   Engine Transaction Completed Aborted Observed Mix (%) \
0     dbfs   NewOrder         0        0           0.000
1     dbfs   Payment         0        0           0.000
2     dbfs OrderStatus         0        0           0.000
3     dbfs   Delivery         0        0           0.000
4     dbfs StockLevel         0        0           0.000
5 PostgreSQL   NewOrder         0        0           0.000

```

6	PostgreSQL	Payment	0	0	0.000
7	PostgreSQL	OrderStatus	0	0	0.000
8	PostgreSQL	Delivery	0	0	0.000
9	PostgreSQL	StockLevel	0	0	0.000

	Abort Rate (%)
0	0.000
1	0.000
2	0.000
3	0.000
4	0.000
5	0.000
6	0.000
7	0.000
8	0.000
9	0.000

```
[7]: fig, axes = plt.subplots(1, 2, figsize=(14, 5))

for engine, run in runs.items():
    results_df = run['results_df']
    label = run['display_name']
    axes[0].plot(
        results_df['Time (seconds)'],
        results_df['Throughput (requests/second)'],
        marker='o',
        label=label,
    )
    axes[1].plot(
        results_df['Time (seconds)'],
        results_df['95th Percentile Latency (millisecond)'],
        marker='o',
        label=label,
    )

axes[0].axhline(
    summary_comparison_df.loc['dbfs', 'Throughput (req/s)'],
    linestyle='--',
    linewidth=1,
    label='dbfs overall avg',
)
axes[0].axhline(
    summary_comparison_df.loc['PostgreSQL', 'Throughput (req/s)'],
    linestyle=':',
    linewidth=1,
    label='PostgreSQL overall avg',
)
```

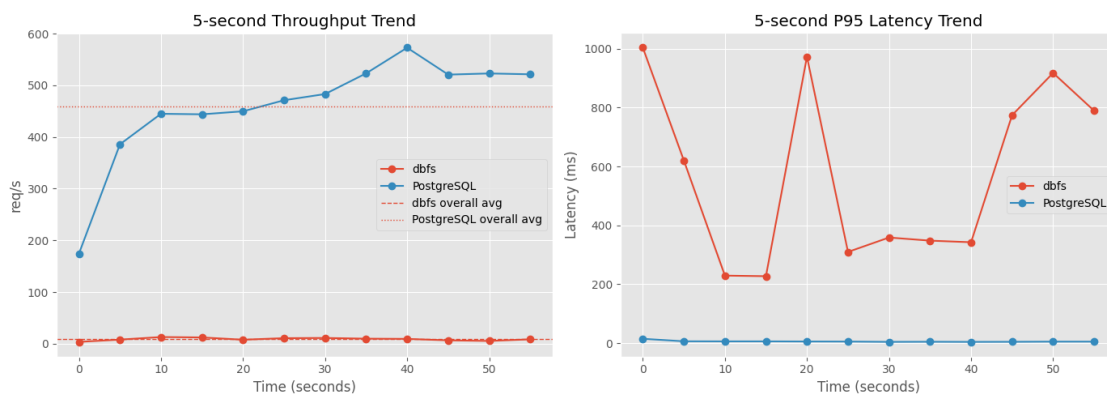
```

axes[0].set_title('5-second Throughput Trend')
axes[0].set_xlabel('Time (seconds)')
axes[0].set_ylabel('req/s')
axes[0].legend()

axes[1].set_title('5-second P95 Latency Trend')
axes[1].set_xlabel('Time (seconds)')
axes[1].set_ylabel('Latency (ms)')
axes[1].legend()

fig.tight_layout()
plt.show()

```



```

[8]: fig, axes = plt.subplots(1, 2, figsize=(14, 5))

for engine, run in runs.items():
    samples_df = run['samples_df'].copy()
    results_df = run['results_df']
    label = run['display_name']
    samples_df['Throughput Rolling'] = samples_df[
        'Throughput (requests/second)'].rolling(5, min_periods=1).mean()

    axes[0].plot(
        samples_df['Time (seconds)'],
        samples_df['Throughput Rolling'],
        linewidth=2,
        label=label,
    )
    axes[1].plot(
        results_df['Time (seconds)'],
        results_df['Median Latency (millisecond)'],
        marker='o',
        alpha=0.8,
    )

```

```

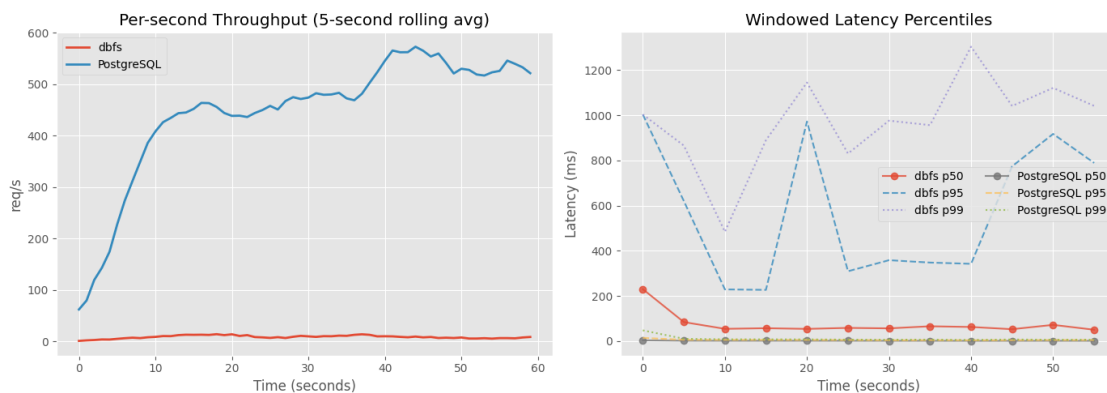
        label=f'{label} p50',
    )
    axes[1].plot(
        results_df['Time (seconds)'],
        results_df['95th Percentile Latency (millisecond)'],
        linestyle='--',
        alpha=0.8,
        label=f'{label} p95',
    )
    axes[1].plot(
        results_df['Time (seconds)'],
        results_df['99th Percentile Latency (millisecond)'],
        linestyle=':',
        alpha=0.8,
        label=f'{label} p99',
    )

axes[0].set_title('Per-second Throughput (5-second rolling avg)')
axes[0].set_xlabel('Time (seconds)')
axes[0].set_ylabel('req/s')
axes[0].legend()

axes[1].set_title('Windowed Latency Percentiles')
axes[1].set_xlabel('Time (seconds)')
axes[1].set_ylabel('Latency (ms)')
axes[1].legend(ncol=2)

fig.tight_layout()
plt.show()

```



```

[9]: procedure_rows = []
    for engine, run in runs.items():
        for procedure_name, procedure_df in run['procedure_dfs'].items():

```

```

        procedure_rows.append({
            'Engine':
                run['display_name'],
            'Procedure':
                procedure_name,
            'Avg Throughput (req/s)':
                procedure_df['Throughput (requests/second)'].mean(),
            'Avg Latency (ms)':
                procedure_df['Average Latency (millisecond)'].mean(),
            'Avg P95 Latency (ms)':
                procedure_df['95th Percentile Latency (millisecond)'].mean(),
        })

procedure_comparison_df = pd.DataFrame(procedure_rows).sort_values(
    ['Procedure', 'Engine'])
display(procedure_comparison_df)

procedure_throughput = procedure_comparison_df.pivot(
    index='Procedure', columns='Engine', values='Avg Throughput (req/s)')
procedure_p95 = procedure_comparison_df.pivot(index='Procedure',
                                                columns='Engine',
                                                values='Avg P95 Latency (ms)')

fig, axes = plt.subplots(1, 2, figsize=(15, 5))
procedure_throughput.plot(kind='bar', ax=axes[0])
axes[0].set_title('Average Throughput by Procedure')
axes[0].set_ylabel('req/s')
axes[0].tick_params(axis='x', rotation=30)

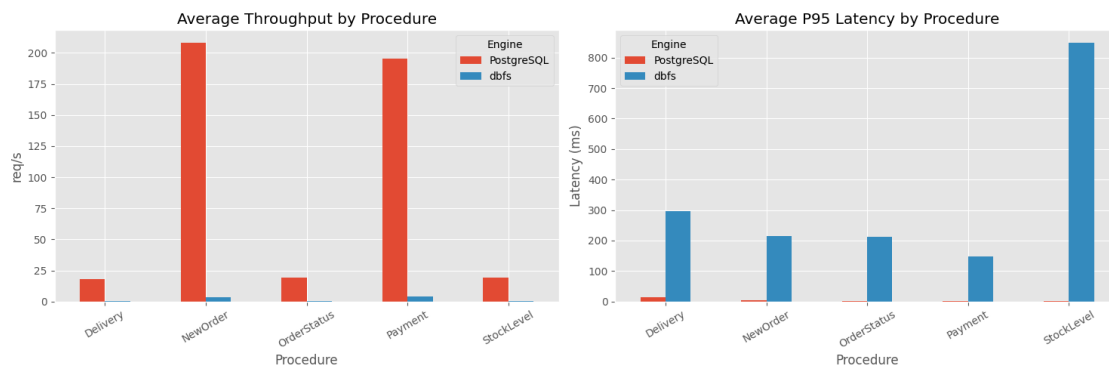
procedure_p95.plot(kind='bar', ax=axes[1])
axes[1].set_title('Average P95 Latency by Procedure')
axes[1].set_ylabel('Latency (ms)')
axes[1].tick_params(axis='x', rotation=30)

fig.tight_layout()
plt.show()

```

	Engine	Procedure	Avg Throughput (req/s)	Avg Latency (ms)	\
5	PostgreSQL	Delivery	17.900	7.633	
0	dbfs	Delivery	0.333	239.931	
6	PostgreSQL	NewOrder	207.833	3.196	
1	dbfs	NewOrder	3.750	99.147	
7	PostgreSQL	OrderStatus	19.183	1.237	
2	dbfs	OrderStatus	0.500	170.564	
8	PostgreSQL	Payment	195.350	1.247	
3	dbfs	Payment	3.900	49.682	
9	PostgreSQL	StockLevel	19.100	0.971	
4	dbfs	StockLevel	0.383	713.732	

	Avg P95 Latency (ms)
5	13.349
0	297.642
6	5.384
1	215.784
7	2.862
2	212.293
8	2.265
3	148.369
9	1.598
4	847.614



```
[10]: mix_comparison_rows = []
for engine in transaction_mix_df['Engine'].unique():
    engine_mix = transaction_mix_df[transaction_mix_df['Engine'] ==
                                    engine].set_index('Transaction')
    for transaction_name, configured_ratio in configured_ratios.items():
        observed_ratio = engine_mix.loc[transaction_name, 'Observed Mix (%)']
        mix_comparison_rows.append({
            'Engine':
                engine,
            'Transaction':
                transaction_name,
            'Configured Ratio (%)':
                configured_ratio,
            'Observed Ratio (%)':
                observed_ratio,
            'Absolute Error (pp)':
                abs(observed_ratio - configured_ratio),
        })

mix_comparison_df = pd.DataFrame(mix_comparison_rows)
display(mix_comparison_df)
```

```

report_rows = []
for engine, run in runs.items():
    parsed = parse_terminal_log(run['terminal_log'])
    completed_total = transaction_mix_df.loc[transaction_mix_df['Engine'] ==
                                             run['display_name'],
                                             'Completed'].sum()

    aborted_total = transaction_mix_df.loc[
        transaction_mix_df['Engine'] == run['display_name'], 'Aborted'].sum()

    report_rows.append({
        'run_label':
            run['label'],
        'engine':
            run['display_name'],
        'dbms_type':
            parsed['dbms_type'],
        'workload':
            parsed['workload'],
        'scale_factor':
            float(parsed['scale_factor']),
        'terminals':
            int(parsed['terminals']),
        'duration_seconds':
            run['summary']['Elapsed Time (nanoseconds)] / 1_000_000_000,
        'measured_requests':
            run['summary']['Measured Requests'],
        'throughput_req_per_s':
            run['summary']['Throughput (requests/second)'],
        'goodput_req_per_s':
            run['summary']['Goodput (requests/second)'],
        'completed_transactions':
            int(completed_total),
        'aborted_transactions':
            int(aborted_total),
        'abort_percentage':
            (aborted_total / (completed_total + aborted_total) * 100) if
            (completed_total + aborted_total) else 0.0,
    })

report_df = pd.DataFrame(report_rows)
display(report_df)

```

	Engine	Transaction	Configured Ratio (%)	Observed Ratio (%) \
0	dbfs	NewOrder	45.000	0.000
1	dbfs	Payment	43.000	0.000
2	dbfs	OrderStatus	4.000	0.000
3	dbfs	Delivery	4.000	0.000

4	dbfs	StockLevel	4.000	0.000
5	PostgreSQL	NewOrder	45.000	0.000
6	PostgreSQL	Payment	43.000	0.000
7	PostgreSQL	OrderStatus	4.000	0.000
8	PostgreSQL	Delivery	4.000	0.000
9	PostgreSQL	StockLevel	4.000	0.000

Absolute Error (pp)

0	45.000
1	43.000
2	4.000
3	4.000
4	4.000
5	45.000
6	43.000
7	4.000
8	4.000
9	4.000

	run_label	engine	dbms_type	workload	scale_factor \
0	tpcc_2026-05-30_03-28-55	dbfs	POSTGRES	TPCC	1.000
1	tpcc_2026-05-30_03-31-38	PostgreSQL	POSTGRES	TPCC	1.000

	terminals	duration_seconds	measured_requests	throughput_req_per_s \
0	1	60.000	532	8.867
1	1	60.000	27562	459.366

	goodput_req_per_s	completed_transactions	aborted_transactions \
0	8.133	0	0
1	457.433	0	0

abort_percentage

0	0.000
1	0.000

```
[11]: def summarize_dbfs_query_trace(trace_df):
    summary_df = trace_df.copy()
    summary_df['sample_sql'] = summary_df['sample_sql'].fillna(
        summary_df['shape_key'])
    summary_df['total_elapsed_ms'] = summary_df['total_elapsed_us'] / 1000.0
    summary_df['mean_elapsed_ms'] = summary_df['mean_elapsed_us'] / 1000.0
    summary_df['p95_elapsed_ms'] = pd.NA
    summary_df['max_elapsed_ms'] = summary_df['max_elapsed_us'] / 1000.0
    summary_df = summary_df.sort_values(['total_elapsed_ms', 'executions'],
                                         ascending=[False, False])

    return summary_df[[
        'kind',
        'shape_key',
```

```

        'sample_sql',
        'executions',
        'total_elapsed_ms',
        'mean_elapsed_ms',
        'p95_elapsed_ms',
        'max_elapsed_ms',
        'failures',
    ]]

def summarize_postgres_server_query_trace(trace_df):
    summary_df = trace_df.copy()
    summary_df['kind'] = 'QUERY'
    summary_df['shape_key'] = summary_df['query'].str.replace(
        r'\s+', ' ', regex=True).str.strip()
    summary_df['sample_sql'] = summary_df['query']
    summary_df['executions'] = summary_df['calls']
    summary_df['total_elapsed_ms'] = summary_df['total_exec_time']
    summary_df['mean_elapsed_ms'] = summary_df['mean_exec_time']
    summary_df['p95_elapsed_ms'] = pd.NA
    summary_df['max_elapsed_ms'] = summary_df['max_exec_time']
    summary_df['failures'] = 0
    summary_df = summary_df.sort_values(['total_elapsed_ms', 'executions'],
                                         ascending=[False, False])

    return summary_df[[
        'kind',
        'shape_key',
        'sample_sql',
        'executions',
        'total_elapsed_ms',
        'mean_elapsed_ms',
        'p95_elapsed_ms',
        'max_elapsed_ms',
        'failures',
    ]]

def summarize_postgres_query_trace(trace_df):
    if {
        'query', 'calls', 'total_exec_time', 'mean_exec_time',
        'max_exec_time'
    }.issubset(trace_df.columns):
        return summarize_postgres_server_query_trace(trace_df)
    return summarize_dbfs_query_trace(trace_df)

def escape_plot_label(text):

```

```

return text.replace('$', r'\$')

def build_shared_trace_comparison(trace_summaries):
    if not {'dbfs', 'postgres'}.issubset(trace_summaries):
        return None

    shared_df = trace_summaries['dbfs'].merge(
        trace_summaries['postgres'],
        on='shape_key',
        how='inner',
        suffixes=('_dbfs', '_postgres'),
    )
    if shared_df.empty:
        return shared_df

    shared_df['sample_sql'] = shared_df['sample_sql_dbfs'].fillna(
        shared_df['sample_sql_postgres'])
    shared_df['combined_total_elapsed_ms'] = (
        shared_df['total_elapsed_ms_dbfs'] +
        shared_df['total_elapsed_ms_postgres'])
    shared_df['combined_max_elapsed_ms'] = shared_df[[
        'max_elapsed_ms_dbfs',
        'max_elapsed_ms_postgres',
    ]].max(axis=1)
    shared_df['label'] = shared_df['sample_sql'].str.slice(0, 96)
    shared_df['plot_label'] = shared_df['label'].map(escape_plot_label)
    return shared_df.sort_values(
        ['combined_total_elapsed_ms', 'combined_max_elapsed_ms'],
        ascending=[False, False],
    )

trace_summaries = {}
for engine, run in runs.items():
    trace_df = run['query_trace_df']
    if trace_df is None or trace_df.empty:
        print(
            f"{run['display_name']}: {query_trace_filenames[engine]} was not_
↳ found."
        )
        continue

    if engine == 'dbfs':
        engine_summary_df = summarize_dbfs_query_trace(trace_df.copy())
    else:
        engine_summary_df = summarize_postgres_query_trace(trace_df.copy())

```

```

engine_summary_df['Engine'] = run['display_name']
engine_summary_df['time_share_pct'] = (
    engine_summary_df['total_elapsed_ms'] /
    engine_summary_df['total_elapsed_ms'].sum() * 100)
engine_summary_df['label'] = engine_summary_df['sample_sql'].str.slice(
    0, 96)
engine_summary_df['plot_label'] = engine_summary_df['label'].map(
    escape_plot_label)
trace_summaries[engine] = engine_summary_df

for engine, summary_df in trace_summaries.items():
    display_name = runs[engine]['display_name']
    print(f'Top query shapes for {display_name}')
    display(summary_df[[
        'kind',
        'executions',
        'time_share_pct',
        'total_elapsed_ms',
        'mean_elapsed_ms',
        'p95_elapsed_ms',
        'max_elapsed_ms',
        'failures',
        'sample_sql',
    ]].head(15))

shared_trace_df = build_shared_trace_comparison(trace_summaries)
if shared_trace_df is not None:
    overlap_summary_df = pd.DataFrame([{
        'dbfs_shapes':
            len(trace_summaries['dbfs']),
        'postgres_shapes':
            len(trace_summaries['postgres']),
        'shared_shapes':
            len(shared_trace_df),
        'dbfs_only':
            len(trace_summaries['dbfs']) - len(shared_trace_df),
        'postgres_only':
            len(trace_summaries['postgres']) - len(shared_trace_df),
    }])
    print('Shared query shape overlap')
    display(overlap_summary_df)

    display(shared_trace_df[[
        'kind_dbfs',
        'executions_dbfs',
        'executions_postgres',
    ]])

```

```

        'total_elapsed_ms_dbfs',
        'total_elapsed_ms_postgres',
        'max_elapsed_ms_dbfs',
        'max_elapsed_ms_postgres',
        'sample_sql',
    ]].head(15))

fig, axes = plt.subplots(1, 2, figsize=(16, 8))

plot_specs = [
    (
        'combined_total_elapsed_ms',
        'total_elapsed_ms',
        'Shared Query Shapes by Total Time',
    ),
    (
        'combined_max_elapsed_ms',
        'max_elapsed_ms',
        'Shared Query Shapes by Max Latency',
    ),
]

for axis, (combined_metric, metric, title) in zip(axes, plot_specs):
    top_df = shared_trace_df.nlargest(
        10, combined_metric).sort_values(combined_metric)
    positions = list(range(len(top_df)))
    axis.barh(
        [position - 0.2 for position in positions],
        top_df[f'{metric}_dbfs'],
        height=0.4,
        label='dbfs',
        alpha=0.8,
    )
    axis.barh(
        [position + 0.2 for position in positions],
        top_df[f'{metric}_postgres'],
        height=0.4,
        label='PostgreSQL',
        alpha=0.8,
    )
    axis.set_yticks(positions)
    axis.set_yticklabels(top_df['plot_label'])
    axis.set_title(title)
    axis.set_xlabel('Latency (ms)')

axes[0].legend()
fig.tight_layout()

```

```
plt.show()
```

Top query shapes for dbfs

	kind	executions	time_share_pct	total_elapsed_ms	mean_elapsed_ms	\
0	QUERY	23	33.773	19,358.632	841.679	
1	QUERY	158	12.342	7,074.403	44.774	
2	QUERY	2279	8.876	5,087.783	2.232	
3	UPDATE	20	8.644	4,954.498	247.724	
4	QUERY	2281	8.321	4,769.666	2.091	
5	QUERY	30	5.897	3,380.277	112.675	
6	UPDATE	2262	5.429	3,112.130	1.375	
7	UPDATE	2262	3.195	1,831.257	0.809	
8	QUERY	20	1.949	1,117.057	55.852	
9	QUERY	226	1.933	1,107.871	4.902	
10	UPDATE	234	0.988	566.454	2.420	
11	QUERY	234	0.984	563.907	2.409	
12	QUERY	234	0.959	549.687	2.349	
13	UPDATE	234	0.845	484.450	2.070	
14	QUERY	30	0.837	479.652	15.988	

	p95_elapsed_ms	max_elapsed_ms	failures	\
0	<NA>	1,268.616	0	
1	<NA>	315.115	0	
2	<NA>	92.237	0	
3	<NA>	485.049	0	
4	<NA>	54.415	0	
5	<NA>	418.344	0	
6	<NA>	40.389	0	
7	<NA>	82.585	0	
8	<NA>	102.242	0	
9	<NA>	491.882	0	
10	<NA>	227.021	0	
11	<NA>	68.981	0	
12	<NA>	59.062	0	
13	<NA>	81.936	0	
14	<NA>	48.582	0	

	sample_sql
0	SELECT COUNT(DISTINCT (S_I_ID)) AS STOCK_COUNT...
1	SELECT C_FIRST, C_MIDDLE, C_ID, C_STREET_1, C_...
2	SELECT S_QUANTITY, S_DATA, S_DIST_01, S_DIST_0...
3	UPDATE order_line SET OL_DELIVERY_D = '2026-05...
4	SELECT I_PRICE, I_NAME , I_DATA FROM item WHER...
5	SELECT OL_I_ID, OL_SUPPLY_W_ID, OL_QUANTITY, O...
6	UPDATE stock SET S_QUANTITY = 21 , S_YTD = S_Y...
7	INSERT INTO order_line (OL_O_ID, OL_D_ID, OL_W...
8	SELECT SUM(OL_AMOUNT) AS OL_TOTAL FROM order_l...

```

9  SELECT C_DISCOUNT, C_LAST, C_CREDIT FROM custo...
10 UPDATE district SET D_YTD = D_YTD + 3187.54003...
11 SELECT W_STREET_1, W_STREET_2, W_CITY, W_STATE...
12 SELECT D_STREET_1, D_STREET_2, D_CITY, D_STATE...
13 UPDATE warehouse SET W_YTD = W_YTD + 3187.5400...
14 SELECT O_ID, O_CARRIER_ID, O_ENTRY_D FROM oord...

```

Top query shapes for PostgreSQL

	kind	executions	time_share_pct	total_elapsed_ms	mean_elapsed_ms	\
0	UPDATE	300040	22.866	12,662.712	0.042	
1	QUERY	124955	11.643	6,447.562	0.051	
2	QUERY	125072	10.634	5,888.966	0.047	
3	UPDATE	123921	8.231	4,558.264	0.036	
4	QUERY	7613	5.261	2,913.205	0.382	
5	UPDATE	100000	4.700	2,602.879	0.026	
6	UPDATE	100000	3.736	2,068.882	0.020	
7	UPDATE	123921	3.282	1,817.580	0.014	
8	UPDATE	12471	1.610	891.657	0.071	
9	UPDATE	10740	1.559	863.498	0.080	
10	QUERY	10740	1.510	835.930	0.077	
11	UPDATE	11721	1.471	814.449	0.069	
12	UPDATE	10740	1.445	800.376	0.074	
13	QUERY	12471	1.433	793.839	0.063	
14	QUERY	1146	1.424	788.485	0.688	

	p95_elapsed_ms	max_elapsed_ms	failures	\
0	<NA>	0.839	0	
1	<NA>	27.249	0	
2	<NA>	39.915	0	
3	<NA>	0.910	0	
4	<NA>	8.502	0	
5	<NA>	1.715	0	
6	<NA>	0.270	0	
7	<NA>	0.973	0	
8	<NA>	9.383	0	
9	<NA>	3.478	0	
10	<NA>	1.888	0	
11	<NA>	31.315	0	
12	<NA>	6.310	0	
13	<NA>	14.714	0	
14	<NA>	10.540	0	

```

                                sample_sql
0  INSERT INTO order_line VALUES (1, 1, 1, 1, 592...
1  SELECT S_QUANTITY, S_DATA, S_DIST_01, S_DIST_0...
2  SELECT I_PRICE, I_NAME , I_DATA FROM item WHER...
3  INSERT INTO order_line (OL_O_ID, OL_D_ID, OL_W...
4  SELECT C_FIRST, C_MIDDLE, C_ID, C_STREET_1, C_...

```

```

5  INSERT INTO item VALUES (1, 'sxvnjhpqdxvcra...
6  INSERT INTO stock VALUES (1, 1, 84, 0.0, 0, 0,...
7  UPDATE stock SET S_QUANTITY = 44 , S_YTD = S_Y...
8  INSERT INTO new_order (NO_O_ID, NO_D_ID, NO_W_...
9  UPDATE order_line SET OL_DELIVERY_D = '2026-05...
10 SELECT O_C_ID FROM oorder WHERE O_ID = 2101 AN...
11 UPDATE warehouse SET W_YTD = W_YTD + 492.04000...
12 UPDATE oorder SET O_CARRIER_ID = 1 WHERE O_ID ...
13 SELECT C_DISCOUNT, C_LAST, C_CREDIT FROM custo...
14 SELECT COUNT(DISTINCT (S_I_ID)) AS STOCK_COUNT...

```

Shared query shape overlap

	dbfs_shapes	postgres_shapes	shared_shapes	dbfs_only	postgres_only
0	30	69	30	0	39

	kind_dbfs	executions_dbfs	executions_postgres	total_elapsed_ms_dbfs	\
0	QUERY	23	1146	19,358.632	
2	QUERY	2279	124955	5,087.783	
4	QUERY	2281	125072	4,769.666	
1	QUERY	158	7613	7,074.403	
7	UPDATE	2262	123921	1,831.257	
3	UPDATE	20	10740	4,954.498	
6	UPDATE	2262	123921	3,112.130	
5	QUERY	30	1151	3,380.277	
9	QUERY	226	12471	1,107.871	
8	QUERY	20	10740	1,117.057	
13	UPDATE	234	11721	484.450	
10	UPDATE	234	11721	566.454	
11	QUERY	234	11721	563.907	
16	QUERY	226	12471	476.698	
12	QUERY	234	11721	549.687	

	total_elapsed_ms_postgres	max_elapsed_ms_dbfs	max_elapsed_ms_postgres	\
0	788.485	1,268.616	10.540	
2	6,447.562	92.237	27.249	
4	5,888.966	54.415	39.915	
1	2,913.205	315.115	8.502	
7	4,558.264	82.585	0.910	
3	863.498	485.049	3.478	
6	1,817.580	40.389	0.973	
5	114.858	418.344	2.752	
9	793.839	491.882	14.714	
8	529.110	102.242	5.027	
13	814.449	81.936	31.315	
10	611.984	227.021	5.129	
11	543.327	68.981	12.633	
16	597.917	18.175	7.666	
12	513.416	59.062	4.129	

```

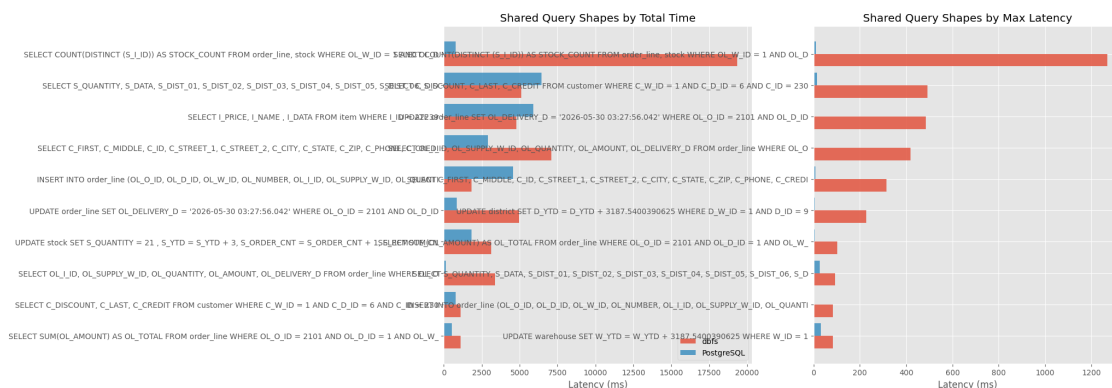
                                sample_sql
0  SELECT COUNT(DISTINCT (S_I_ID)) AS STOCK_COUNT...
2  SELECT S_QUANTITY, S_DATA, S_DIST_01, S_DIST_0...
4  SELECT I_PRICE, I_NAME , I_DATA FROM item WHER...
1  SELECT C_FIRST, C_MIDDLE, C_ID, C_STREET_1, C_...
7  INSERT INTO order_line (OL_O_ID, OL_D_ID, OL_W...
3  UPDATE order_line SET OL_DELIVERY_D = '2026-05...
6  UPDATE stock SET S_QUANTITY = 21 , S_YTD = S_Y...
5  SELECT OL_I_ID, OL_SUPPLY_W_ID, OL_QUANTITY, O...
9  SELECT C_DISCOUNT, C_LAST, C_CREDIT FROM custo...
8  SELECT SUM(OL_AMOUNT) AS OL_TOTAL FROM order_l...
13 UPDATE warehouse SET W_YTD = W_YTD + 3187.5400...
10 UPDATE district SET D_YTD = D_YTD + 3187.54003...
11 SELECT W_STREET_1, W_STREET_2, W_CITY, W_STATE...
16      SELECT W_TAX FROM warehouse WHERE W_ID = 1
12 SELECT D_STREET_1, D_STREET_2, D_CITY, D_STATE...

```

/var/folders/fb/ffh7t7rs47jgclbwtxyfr75w0000gn/T/ipykernel_88041/379990782.py:20

0: UserWarning: Tight layout not applied. tight_layout cannot make Axes width small enough to accommodate all Axes decorations

fig.tight_layout()



1.1 Comparison Notes

Conclusion template:

- PostgreSQL shows a large throughput advantage over dbfs under the same TPCC configuration.
- dbfs completed the same workload shape without unexpected SQL errors, which means the current SQL support is sufficient for this TPCC slice.
- The main next step is performance-oriented investigation: loader behavior, per-procedure latency spikes, and the gap between dbfs and PostgreSQL throughput.
- If `dbfs_query_trace.csv` and `postgres_query_trace.csv` are present in the result directories, the notebook also ranks query shapes by total time, mean latency, and max latency

to highlight bottlenecks on each engine.